

HW05. Báo cáo Performance Testing

Họ tên	Nguyễn Thành Dân
MSSV	23127334
Repo	https://github.com/ThanhDang-Vn/software-testing
SUT	EShop backend (Node.js + SQLite), http://localhost:3000
Công cụ đo	JMeter 5.6.3, Java 17.0.12
AI hỗ trợ	Claude Opus 4.8 (Claude Code), khai đầy đủ ở report/ai-audit-report.md
Máy chạy	hostname Tony , Intel Core i7-12700H (14 nhân / 20 luồng), RAM ~40GB, Windows 11 build 10.0.26200, Node v20.20.2
Ngày đo	2026-08-11

Báo cáo này viết để đọc độc lập là hiểu được. Số liệu đều lấy trực tiếp từ raw [.jtl](#). Những chỗ cần xem sâu thì mình dẫn link tới file trong [docs/](#) và [results/](#) thay vì chép lại nguyên.

Mục lục

1. Phạm vi và cách chọn workflow
2. Môi trường và cách chạy
3. Xác nhận contract bằng smoke test
4. Thiết kế test plan
5. Tham số tải ba scenario
6. Review test plan do AI sinh
7. Thực thi, reset khóa tài khoản, bằng chứng
8. Kết quả ba scenario
9. Endurance và ngưỡng phần cứng
10. Phân tích bằng AI và phần soi lại
11. Bug ghi nhận
12. Đề xuất kiểm thử hiệu năng liên tục
13. Agent Skill
14. Kết luận và hạn chế

1. Phạm vi và cách chọn workflow

Đề yêu cầu đo ba nhóm endpoint: auth-heavy, read-heavy, transactional, và cả ba test plan Load/Stress/Spike phải chạy chung một workflow đi từ đầu tới cuối. Mình chọn luồng mua hàng của một người dùng thật, vì nó tự nhiên chạm đủ ba nhóm trong một mạch:

```
login    -> search sản phẩm  -> xem chi tiết  -> thêm giỏ  -> checkout
(auth)   (read)              (read)           (write)    (write)
```

Ánh xạ sang ba nhóm:

Nhóm	Endpoint trong workflow	Vì sao tạo tải cho nhóm đó
Auth-heavy	<code>POST /api/login</code>	Mỗi vòng lặp ký một JWT và SELECT bảng users theo email.
Read-heavy	<code>GET /api/products?search=</code> và <code>GET /api/products/:id</code>	Hai trong năm request là đọc DB, đúng kiểu thương mại điện tử: đọc nhiều hơn ghi.
Transactional	<code>POST /api/cart</code> và <code>POST /api/checkout</code>	Ghi trạng thái. Checkout INSERT vào bảng orders của SQLite, chính là chỗ nghẽn.

Tỉ lệ mỗi vòng lặp là một auth, hai read, hai write. Vì cả ba scenario dùng chung workflow này, khi đổi số người dùng ảo và pattern thì cả ba nhóm cùng bị nén tải theo, nên so ba scenario với nhau là công bằng. Thiết kế đầy đủ ở `docs/p1-workflow-design.md`.

2. Môi trường và cách chạy

SUT là Node.js một tiến trình, event-loop đơn luồng, dùng thư viện `sqlite3` với một file `database.sqlite`. Đặc điểm này quyết định gần như mọi kết luận về sau: SQLite khóa ghi toàn file, nên các lệnh checkout ghi tuần tự, và Node chỉ có một luồng xử lý.

Quy trình chạy một scenario:

```
node server.js          # server reseed DB khi khởi động
node data/register-users.js  # nạp 300 tài khoản perf
jmeter -n -t <plan>.jmx -l results/jtl/<S>.jtl -e -o results/html/<S>
```

Toàn bộ ba scenario chạy bằng `results/run-all.sh`.

3. Xác nhận contract bằng smoke test

Trước khi tin tài liệu API, mình gọi thử từng endpoint bằng `curl` (chi tiết ở `docs/p0-smoke-test.md`). Vài điểm quan trọng rút ra và ảnh hưởng tới thiết kế:

Phát hiện	Ảnh hưởng
Login trả field <code>token</code> (JWT payload <code>{id, role}</code>)	Extract <code>token</code> để gắn Bearer cho cart và checkout.
Khóa tài khoản lịch tài liệu: code cộng <code>login_attempts</code> thêm 2 mỗi lần sai, khóa khi đạt 3	Thực tế khóa chỉ sau 2 lần sai, giữ 3 phút. Dùng credential hợp lệ thì login thành công và attempts reset về 0, nên khóa không kích hoạt.
Detail của id chặn trả <code>price</code> kiểu string; id không tồn tại trả <code>{}</code> kèm 200	Assertion phần detail bám theo <code>name</code> , không assert <code>price</code> .
Cart lưu in-memory theo userId, checkout ghi bảng orders	Mỗi người dùng ảo cần một tài khoản riêng để không dồn chung một giỏ.

4. Thiết kế test plan

4.1 Năm bước và cách kiểm

Bước	Request	Trích / correlation	Assertion	Think-time
1 Login	<code>POST /api/login</code>	<code>authToken</code> từ <code>\$.token</code>	code 200, body có <code>token</code>	2 tới 5s
2 Search	<code>GET /api/products?search=\${search_keyword}</code>	<code>found_id</code> từ <code>\$.id</code> , default <code>\$.product_id</code>	code 200, body là mảng	3 tới 8s
3 Detail	<code>GET /api/products/\${found_id}</code>		code 200, body chứa <code>name</code>	5 tới 12s
4 Add-to-cart	<code>POST /api/cart</code> (Bearer)		code 200, body có <code>Added to cart</code>	2 tới 5s
5 Checkout	<code>POST /api/checkout</code> (Bearer)	<code>orderId</code> từ <code>\$.orderId</code>	code 200, body có <code>Checkout successful</code>	1 tới 3s

4.2 Xử lý lỗi thống nhất

- Thread Group đặt on sample error là continue, một request lỗi không giết cả người dùng ảo.

- Bọc bước 4 và 5 trong If Controller kiểm tra `authToken` khác rỗng, để lúc login lỗi không bắn ra loạt 401 vô nghĩa làm nhiều error%.
- Bộ trích có default value, nên search lỗi vẫn còn `product_id` từ CSV để đi tiếp.

4.3 Data-driven

Dữ liệu tham số hoá ở `data/`. File `users.csv` có 300 tài khoản, đủ để tải đỉnh 300 người dùng ảo mỗi người một tài khoản riêng. Lý do cần đủ tài khoản (tránh khóa tài khoản, tránh tranh chấp giả trên cùng một giỏ, và sát thực tế nhiều người dùng khác nhau) giải thích ở `data/README.md`.

Ba plan dùng chung một khối workflow đặt ở `testplans/_workflow-fragment.xml`, và một script ráp khối đó thành ba file, nên workflow của ba plan giống hệt nhau tới từng byte, chỉ khác profile tải và listener. Tên plan theo quy ước: `23127334_{Load,Stress,Spike}_20260811.jmx`.

5. Tham số tải ba scenario

Chi tiết và lý do ở `docs/p1-load-params.md`. Điểm cần nói: với think-time thật (khoảng 23 giây một vòng lặp), 300 người dùng ảo mới cho khoảng 65 req/s, nhiều khả năng chưa đủ đẩy server tới ngưỡng. Vì vậy mình tham số hoá think-time bằng property `tt_mult`. Chuỗi request, correlation, assertion giữ nguyên; chỉ hệ số think-time đổi theo scenario.

Thuộc tính	Load	Stress	Spike
Mục tiêu	tải điển hình ổn định	tìm ngưỡng gãy	chịu và phục hồi sau đột biến
Người dùng ảo	50 phẳng	50 tới 300 theo 6 bậc, mỗi bậc thêm 50 giữ 60s	nền 10, bơm lên 300 trong 5s, rồi về 10
Think-time	thật 100%	nén còn 0.3	nén còn 0.3
Thời lượng	5 phút	khoảng 7 phút	khoảng 4 phút
Listener	Summary Report	Aggregate Report	View Results Tree

Ba listener khác nhau đúng yêu cầu không lặp loại report view.

6. Review test plan do AI sinh

Mình để AI dựng workflow và sinh JMX theo từng bước, rồi tự soi lại như một người review độc lập (chi tiết ở `docs/p1-human-review.md`). Mấy chỗ AI làm chưa đạt và mình phải sửa:

Chỗ AI làm chưa đạt	Vì sao AI trượt	Mình sửa thành
Bản Stress chỉ là một Thread Group ramp tuyến tính 300 threads trong 300s, không có plateau từng bậc	AI đọc "tăng dần" thành ramp trơn, không nghĩ tới việc cần cửa sổ giữ tải để quy p95 cho từng mức	6 bậc, mỗi bậc thêm 50 người dùng ảo, giữ 60s
Think-time để cứng, không nén được	Prompt chưa nói tới nhu cầu nén think để chạm ngưỡng trên phần cứng yếu	Tham số hoá bằng property <code>tt_mult</code>
Assertion ban đầu chỉ kiểm code 200	AI dừng ở mức cú pháp hợp lệ	Thêm kiểm token, kiểm chuỗi đặc trưng, bọc cart/checkout trong If Controller

Điểm mình rút ra: XML hợp lệ và cấu trúc gọn không có nghĩa mô hình tải đúng. Vì ba plan tái dùng một fragment, một lỗi trong fragment nhân sang cả ba, nên review kỹ fragment quan trọng hơn review từng file.

7. Thực thi, reset khóa tài khoản, bằng chứng

Chạy headless bằng `results/run-all.sh`. Trước mỗi lần chạy mình reset theo trình tự: dừng server, khởi động lại, nạp lại 300 tài khoản, rồi mới chạy JMeter. Vì server reseed DB khi khởi động nên cột `login_attempts` và `locked_until` bị xoá sạch, đây cũng chính là cách reset khóa tài khoản mà đề hỏi, đồng thời bảng orders sạch nên mỗi run bắt đầu từ trạng thái giống nhau. Workflow dùng credential hợp lệ nên trong mọi run khóa tài khoản không hề kích hoạt, error% bằng 0 xác nhận điều đó.

Mỗi run xuất một raw `.jtl` ở `results/jtl/` và một thư mục HTML ở `results/html/`. Ảnh terminal JMeter kèm Task Manager tiến trình node trong cùng một khung ở `results/img/` (ba scenario, chụp lúc đang chạy). Hướng dẫn chụp và bảng spec phần cứng ở `evidence/`.

8. Kết quả ba scenario

Số liệu tính từ raw `.jtl` bằng `results/analyze-jtl.js`, đơn vị ms, đầy đủ ở `results/run-summary.md`.

8.1 Bảng tổng

Scenario	Samples	Error%	Avg	p95	p99	Max	Throughput
Load (50 người dùng ảo)	3.599	0%	7	19	29	76	10.1 req/s
Stress (50 tới 300)	46.076	0%	10	32	87	418	124.9 req/s
Spike (10 tới 300 rồi về 10)	13.531	0%	28	106	377	769	55.5 req/s

8.2 Chi tiết theo endpoint (avg / p95, ms)

Bước	Load	Stress	Spike
Login	7 / 16	13 / 40	33 / 118
Search	4 / 9	8 / 29	30 / 119
Detail	4 / 10	9 / 30	25 / 102
Add-to-cart	6 / 12	5 / 13	14 / 49
Checkout	14 / 26	17 / 40	40 / 132

8.3 Nhận xét

- Không run nào có lỗi cứng. Kể cả lúc bơm 300 người dùng ảo, server không crash và không từ chối kết nối.
- Độ trễ tăng rõ theo tải: p95 đi từ 19ms lên 32ms rồi 106ms; max từ 76ms lên 769ms.
- Checkout luôn chậm nhất trong mỗi run (avg 14, 17, 40ms). Đúng như dự đoán từ kiến trúc: checkout ghi bảng orders của SQLite, mà SQLite khóa ghi toàn file.
- Login là bước nhảy tải thứ hai (p95 Spike 118ms, p99 500ms), vì nó vừa đọc DB vừa ký JWT.
- Ở 300 người dùng ảo có think-time vẫn chưa thấy điểm gãy, nên mình chạy thêm endurance để ép tới ngưỡng thật.

9. Endurance và ngưỡng phần cứng

Bài soak: 300 người dùng ảo, bỏ think-time (`tt_mult=0`), giữ 12 phút, đồng thời lấy mẫu RAM tiến trình node mỗi 5 giây. Chi tiết ở [results/endurance/endurance-summary.md](#) .

Chỉ số	Giá trị
Tổng samples	206.816
Max stable throughput	~276 req/s (trung bình cả 12 phút 288 req/s)
Error% suốt 12 phút	0%
Độ trễ tại ngưỡng	avg 1.001ms, p50 1.042ms, p95 1.741ms, p99 2.383ms, max 4.229ms
Tỉ lệ request quá 1 giây	54.4%
Memory ceiling	~117 MB, dao động quanh 100MB, giảm 48 lần trong 132 mẫu nên không rõ rí

Cách đọc ngưỡng: server không gãy bằng lỗi mà bằng độ trễ. Khi ép quá khả năng, Node xếp hàng request thay vì trả lỗi, nên throughput chạm trần khoảng 276 req/s còn độ trễ thì phình to. Có một phút throughput vọt lên 556 req/s rồi tụt, đó là GC xả dồn hàng đợi, không phải năng lực bền vững nên mình không tính là ngưỡng. RAM tăng nhanh lúc khởi động rồi dao động lên xuống, không tăng đơn điệu,

nên kết luận không có rò rỉ trong 12 phút. Khuyến nghị vận hành: giữ tải dưới khoảng 276 req/s để độ trễ còn chấp nhận được.

10. Phân tích bằng AI và phần soi lại

Mình đưa raw `.jtl` cho AI phân tích và đề xuất threshold (bản thô để nguyên ở `docs/p3-ai-analysis.md`), sau đó tự đối chiếu số thật ở `docs/misinterpretation-hunt.md`. AI đọc sai 6 chỗ:

AI kết luận	Số đúng từ raw log
Năng lực tối đa 557 req/s	Bền vững chỉ ~276 req/s; 557 là một bucket nhiều do GC
avg 10ms, thừa mọi SLA	Lúc bão hoà avg thật 1.001ms, p95 1.741ms
RAM 53 lên 107MB là memory leak	RAM dao động, giảm 48 lần, không leak
Spike làm giảm 55% năng lực	Hai run cùng chạm 300 threads và 0% lỗi, throughput khác do concurrency trung bình khác
Load cho thấy trần server 10 req/s	10 req/s bị giới hạn bởi số người dùng ảo và think-time; server thật gánh ~276 req/s
0% error nghĩa là không nghẽn	54.4% request ở endurance vượt 1 giây, nghẽn nằm ở độ trễ

Phần AI đề xuất cách cải thiện, mình phân loại theo kiến trúc thật: bật SQLite WAL, thêm index cho `users.email`, cache read là làm được và đúng bệnh; còn connection pool, chạy nhiều instance dùng chung một file SQLite, và index cho `LIKE '%kw%'` là không dùng được, vì AI áp mô hình DB client-server lên SQLite nhúng.

11. Bug ghi nhận

Từ smoke và quá trình đo, mình ghi nhận các bug thật của SUT. Nội dung đầy đủ và ảnh chụp để post GitHub Issues nằm ở `docs/bug-report.md`.

Bug	Nơi	Mô tả
SQL Injection	<code>server.js</code> search	Tham số <code>search</code> nối thẳng vào <code>LIKE '%...%'</code>
Sai mã trạng thái	product detail	id không tồn tại trả <code>{}</code> kèm 200 thay vì 404
Sai kiểu dữ liệu	product detail	id chặn trả <code>price</code> kiểu string
Lộ thông tin	login	Response trả cả field <code>password</code> dạng plaintext
Sai logic khóa	login	Cộng <code>login_attempts</code> thêm 2 nên khóa chỉ sau 2 lần sai

12. Đề xuất kiểm thử hiệu năng liên tục

Mô hình theo dõi commit, lọc theo đường dẫn để quyết định có chạy, chạy trên một runner cố định, so p95 với baseline và chặn PR khi vượt, nằm ở `docs/continuous-perf-proposal.md`, có sơ đồ và phần bàn về chi phí với báo nhầm. Điểm chính lấy từ bài học đo được: gác bằng p95/p99 chứ không bằng trung bình, và error% một mình là không đủ.

13. Agent Skill

Skill tái dùng cho quy trình này ở `skill/`, gồm SKILL.md mô tả 6 bước, các script generic (sinh plan từ fragment, chạy một scenario, tính p95 từ jtl), một template workflow để đổi cho nhóm endpoint khác, và `EXAMPLE.md` chạy thử trọn nhóm endpoint buyer. Video demo skill mình quay riêng, link dán ở `README.md`.

14. Kết luận và hạn chế

Trên máy `Tony`, EShop backend phục vụ ổn định tới khoảng 276 req/s, không rò rỉ bộ nhớ trong 12 phút, và điểm nghẽn là đường ghi SQLite khi checkout. Bài học lớn nhất ở phần AI: 0% error không đồng nghĩa hệ thống khỏe, và phải nhìn p95/p99 thay vì trung bình.

Hạn chế mình nhìn nhận:

- JMeter chạy cùng máy với SUT, nên hai bên tranh nhau CPU. Con số ngưỡng vì thế là ngưỡng của cả máy chứ không riêng backend. Muốn sạch hơn thì tách JMeter sang máy khác.
- Mỗi scenario chạy một lần, chưa lặp lấy trung vị, nên vẫn dính nhiễu kiểu bucket 556 req/s. Bài endurance đã bù phần này bằng cách nhìn plateau nhiều phút.
- Cửa sổ giữ tải mỗi bậc ở Stress chỉ 60s, hơi ngắn để mỗi mức thật sự ổn định.
- Think-time nên là một lựa chọn để chạm ngưỡng trên phần cứng yếu, không phản ánh nhịp người dùng thật; con số ngưỡng nên hiểu trong bối cảnh đó.

Toàn bộ số trong báo cáo đọc trực tiếp từ raw `.jtl`, có thể tái lập bằng các script trong `results/` và `testplans/`.